AetherFlow: Automated Post-Merge Verification and Dynamic Conflict Resolution in Multi-Agent Programming Environments

A Formal Architecture for Lock-Free Integration and Proactive Remediation at Agent Scale

Samir Baladi

Independent Interdisciplinary Researcher — Ronin Institute / Rite of Renaissance

Research Article — Systems Architecture / Software Engineering / Formal Methods

Research Domain: Source Control, Continuous Integration, Distributed Systems, Agentic Software Engineering

Status: Formal Architecture Proposal with Illustrative Analysis — No Implementation Benchmarked

DOI: https://doi.org/10.5281/zenodo.23979129

ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com

Version 1.0.0 — November 2026

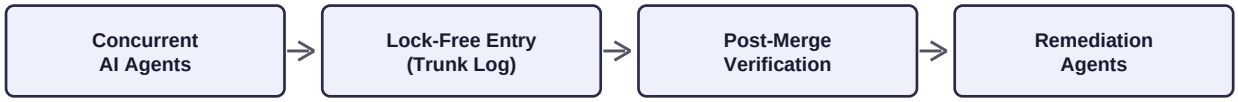
Abstract

When autonomous AI coding agents generate tens of thousands of changesets per day, the serial structure of pre-merge gating — merge queues, branch protection, and global integration locks — becomes the dominant bottleneck of the pipeline. This paper presents AetherFlow, a formal architecture that replaces pre-merge gating with three composable mechanisms: (i) lock-free merge entry, in which changesets are appended to the trunk log without waiting for verification; (ii) asynchronous post-merge verification, computed by a recursive operator that attributes failures to the changeset that introduced them and marks descendants as tainted rather than failed; and (iii) proactive remediation by agents under a graded escalation policy. Three definitions are given: a dynamic conflict window $W(c_i, c_j)$; the recursive verification operator $\oplus_V$; and a bound $P(\text{Remediation}) \geq 1 - (\lambda_{fail}/\mu_{remediate})^K \cdot e^{-\gamma \cdot T_{react}}$ that approaches 1 as remediation tier depth K grows whenever $\lambda_{fail} < \mu_{remediate}$. From these we derive admission latency independent of CI duration, monotonicity of the conflict window, and a closed-form minimum tier depth for any target residual risk, together with a verified-reference discipline that shields consumers from unverified trunk state. AetherFlow is the integration-layer complement to the AetherGit storage architecture. The paper reports no implementation, simulation, or benchmark: all numbers are labeled evaluations of the stated formulas, and an explicit evaluation protocol is given for converting its structural claims into measured ones.

Keywords: *Source Control; Merge Queues; Agentic Software Engineering; Post-Merge Verification; Auto-Remediation; Conflict Resolution; Distributed Systems; Formal Methods; AetherFlow.*

Evidentiary status: *AetherFlow is a proposed formal architecture. No implementation has been built and no simulation or benchmark has been executed for this manuscript.* · ***Illustrative values:*** *every number in Sections 5 and 7 is the evaluation of a stated formula at stated, hypothetical parameters; none is a measurement.* · ***Third-party descriptions*** *of Git, Scion, and Google's infrastructure are drawn from the cited public sources and were not independently verified.*

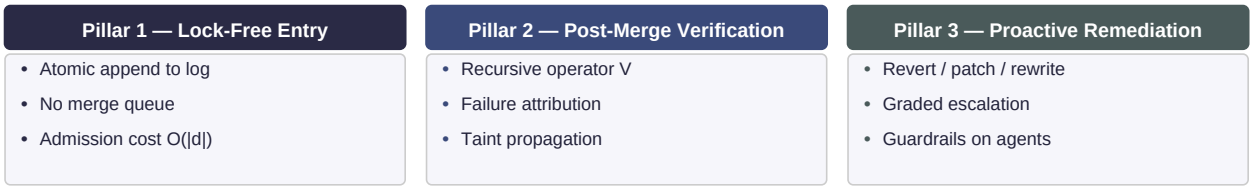
Figure 1



AetherFlow Integration Pipeline

Figure 1. Changesets are admitted to the trunk log immediately; verification and remediation run asynchronously downstream and feed status back to the log. No stage blocks admission.

Figure 2



The Three Architectural Pillars of AetherFlow

Figure 2. The three pillars, each specified formally in Section 3 and architecturally in Section 4.

Table of Contents

1. Introduction	3
2. Background and Related Work	4
3. Formal Framework	5
4. System Architecture	9
5. Illustrative Worked Example (Case Study)	11
6. Comparative Positioning	12
7. Proposed Evaluation Protocol	13
8. Discussion and Threats	14
9. Limitations	15
10. Conclusion and Future Work	15
Appendix A — Notation	16
Appendix B — Derivations	16
References	17
Research Status Statement	18

1. Introduction

1.1 The Integration Bottleneck at Agent Scale

Software integration practice evolved around a human-paced production model: a developer prepares a change, a review and CI cycle validates it, and the change is merged into a shared trunk. The validation step is placed *before* the merge so that the trunk remains releasable. This ordering is sensible when changes arrive at a rate that a verification pipeline can absorb, but it couples two quantities that need not be coupled: the time at which a change becomes part of shared history, and the time at which the change is known to be correct.

Autonomous coding agents change the arrival process. A fleet of agents can generate, test locally, and submit changesets continuously, in parallel, and at a volume that human review capacity cannot scale to match. Three failure modes then emerge from the pre-merge gating model:

- **Merge-queue saturation.** A serial queue whose service time is the CI duration has finite throughput. Once arrivals approach that throughput, waiting time grows without bound (Section 3.4, Figure 6).
- **Pre-merge isolation cost.** Requiring each change to be validated against a fresh trunk state invalidates in-flight validations whenever the trunk advances; the higher the arrival rate, the larger the share of validation work that is discarded and repeated.
- **Conflict-handling overhead.** Many conflicts at agent scale are trivial — a renamed symbol, a changed signature, an overlapping import block — and are precisely the class of defect that an automated agent can diagnose and repair without human attention.

1.2 Relationship to AetherGit and Prior Work

This paper is a companion to AetherGit (Baladi, 2026b), which specifies a *storage-layer* architecture — always-online clients, sparse checkouts, lock-free incremental changesets, and a tiered cache — and which is itself a formal sequel to the author's structural analysis of Anthropic's published account of its Scion source-control system (Anthropic, 2026; Baladi, 2026a). AetherFlow addresses a different layer: what happens *after* a changeset has been created, namely how it is admitted to shared history, how it is verified, and how failures are repaired. The two papers are methodologically independent; AetherFlow requires only that the underlying store expose an append-only changeset log with parent pointers, which AetherGit's changeset tuple provides.

AetherFlow does not claim that post-merge testing is new. Google's infrastructure, for example, couples pre-submit checks with continuous post-submit testing (Potvin & Levenberg, 2016), and speculative merge systems exist in industry (Section 2). The contribution here is a formal treatment that (a) makes the verification result a recursively defined, attributable property of the changeset DAG, (b) bounds the probability that remediation succeeds as a function of explicit capacity parameters, and (c) specifies the guardrails required when the remediating party is itself an autonomous agent.

1.3 Contributions

1. A **lock-free merge-entry model** and a proof that admission latency is independent of CI duration (Proposition 1).
2. A **dynamic conflict window** $W(c_i, c_j)$ with proven monotonicity properties (Definition 1, Proposition 2).
3. A **recursive verification operator** with three-valued status — verified, failed, tainted — that separates the changeset that introduced a failure from its descendants (Definition 2).

DOI: https://doi.org/10.5281/zenodo.23979129

4 An **auto-remediation bound** and a closed-form minimum capacity tier depth for a target residual risk (Definition 3, Proposition 3).

5 A **verified-reference consumption discipline** that confines the risk of lock-free entry to the trunk log, together with a graded remediation policy and guardrails for agent-performed repair (Section 4).

6 An **illustrative case study**, a **comparative positioning**, and a **concrete evaluation protocol** (Sections 5–7).

This paper does not report an implementation, simulation, or benchmark. Where numerical values appear, they are the output of evaluating the stated formulas at stated hypothetical parameters, and are labeled as such.

2. Background and Related Work

2.1 Continuous Integration and Merge Queues

Continuous integration, in the formulation popularized by Fowler (2006), asks developers to integrate frequently and to verify every integration automatically. Hosted platforms operationalize the idea with branch protection and merge queues that serialize merges and re-run checks against the projected post-merge state. The queue guarantees that the trunk never contains a change that failed its checks in the context of the changes ahead of it, at the cost of latency that grows with queue depth.

Industrial work has attacked the serialization cost from the inside. Ananthanarayanan et al. (2019) describe a system that keeps a mainline green at scale by *speculatively* building multiple orderings of pending changes and predicting which will succeed; the approach preserves the always-green invariant while increasing parallelism. AetherFlow takes the opposite structural decision: rather than making the pre-merge gate faster, it relaxes the invariant on the *trunk log* and re-establishes it on a separate verified reference (Section 4.3).

2.2 Post-Submit Testing and Monorepo Infrastructure

Potvin and Levenberg (2016) describe a single-repository model in which changes are reviewed, checked, and submitted, with a test automation platform continuously executing affected tests after submission and surfacing breakages. Meta's Sapling and Mononoke (Sapling SCM, n.d.) similarly pair a scalable server-side store with large-repository tooling. These systems establish that post-submission verification is compatible with very large engineering organizations; they were designed around human submitters and human triage. Anthropic's account of Scion (Anthropic, 2026) shows the shift to agentic workloads for the storage layer; the integration layer for such workloads is the subject of the present paper.

2.3 Optimistic Concurrency, Lock-Free Progress, and Compensation

Lock-free entry has a long theoretical lineage. Kung and Robinson (1981) formalized optimistic concurrency control: execute without locks, validate at commit, and abort on conflict. Herlihy (1991) established wait-free and lock-free synchronization as a hierarchy of progress guarantees, showing that appropriately chosen atomic primitives allow every participant to make progress without mutual exclusion. AetherFlow's admission step uses an atomic append to a reference log, which provides lock-free progress for submitters. Where optimistic control aborts at commit, AetherFlow defers detection and handles failure by *compensation*: the saga model of Garcia-Molina and Salem (1987) represents a long-running activity as a sequence of steps each paired with a compensating action. Revert and corrective patch in Section 4.4 are compensating actions in exactly this sense.

2.4 Semantic Conflicts and Test Reliability

Textual merge success does not imply behavioral correctness. Brun et al. (2011) show that conflicts can be detected proactively and that some arise only at the semantic level, which line-based merging cannot see. Consequently, post-merge verification must run builds and tests, not merely textual checks. Luo et al. (2014) document that flaky tests are common and have identifiable root causes; because AetherFlow attributes failures from CI results (Definition 2), flakiness is a first-class threat and is treated in Section 8.

2.5 Automated Repair and Agentic Software Engineering

Automated program repair has an established research line (Le Goues et al., 2012), and evaluation on real repository issues has become a standard way to measure agent capability (Jimenez et al., 2024). The remediation agents of Pillar 3 draw on this capability but operate under a narrower contract than general issue resolution: the failing changeset, the failing tests, and the pre-failure verified state are all available, which constrains the search space for a corrective patch.

2.6 Queueing Theory

The analysis of the serial merge queue relies on standard queueing results (Kleinrock, 1975) and on Little's law (Little, 1961), which relates mean population, arrival rate, and mean time in system for any stable system. These tools make the saturation argument in Section 3.4 a matter of classical analysis rather than of assertion.

2.7 Research Gap

The literature establishes (a) that serialization is costly, (b) that speculative parallelism can mitigate it, (c) that post-submit testing scales, and (d) that agents can repair code. What it does not provide is a single formal object that states, for an agentic workload, how verification status propagates over the changeset DAG, how likely automated remediation is to succeed as a function of capacity, and what the consumer-facing invariant is when the trunk log is allowed to be temporarily unverified. AetherFlow is a proposal to fill that gap.

3. Formal Framework

We model integration as a continuous stream of changesets rather than as discrete, isolated transactions. Changesets are those of AetherGit (Baladi, 2026b): $c_i = (\delta_i, P(c_i), \tau_i, \sigma_i)$, where δ_i is the content delta, $P(c_i)$ the set of parent changesets, τ_i a timestamp, and σ_i a content-addressed signature. Only the delta, parent set, and timestamp are used below.

3.1 Dynamic Conflict Window (Definition 1)

Rather than blocking concurrent writes, AetherFlow estimates the probability that two parallel changesets conflict:

$$W(c_i, c_j) = P(\text{Conflict}) = f(\ \tau(c_i, c_j),\ \delta_i,\ \delta_j,\ G\ ) \tag{1}$$

where $\tau(c_i, c_j) = |\tau_i - \tau_j|$ is the temporal distance between injections, δ_i and δ_j are the deltas (file footprints and line ranges), and G is the dependency graph of the codebase. The function f is left abstract in the definition; the only requirements are that f is non-increasing in τ and non-decreasing in footprint overlap and in dependency proximity. Intuitively, two edits injected seconds apart to nearby code are likely to be mutually unaware; the same edits injected hours apart are likely to have been made against each other's results.

DOI: https://doi.org/10.5281/zenodo.23979129

An admissible instantiation. Define an overlap score $O(\delta_i, \delta_j, G) \in [0, 1]$ combining the Jaccard overlap of file footprints with a decay in dependency-graph distance d_G between the touched symbols, for example $O = \max(J(F_i, F_j), e^{-d_G/L})$ with characteristic length L. Then take

$$W = 1 - \exp(-\kappa \cdot O \cdot e^{-\tau/\theta}) \tag{1a}$$

with sensitivity $\kappa > 0$ and decay horizon $\theta > 0$. This is the form plotted in Figure 3. It satisfies the stated requirements and takes values in [0, 1): W = 0 when O = 0, and W approaches 1 only for large $\kappa \cdot O$ at small τ.

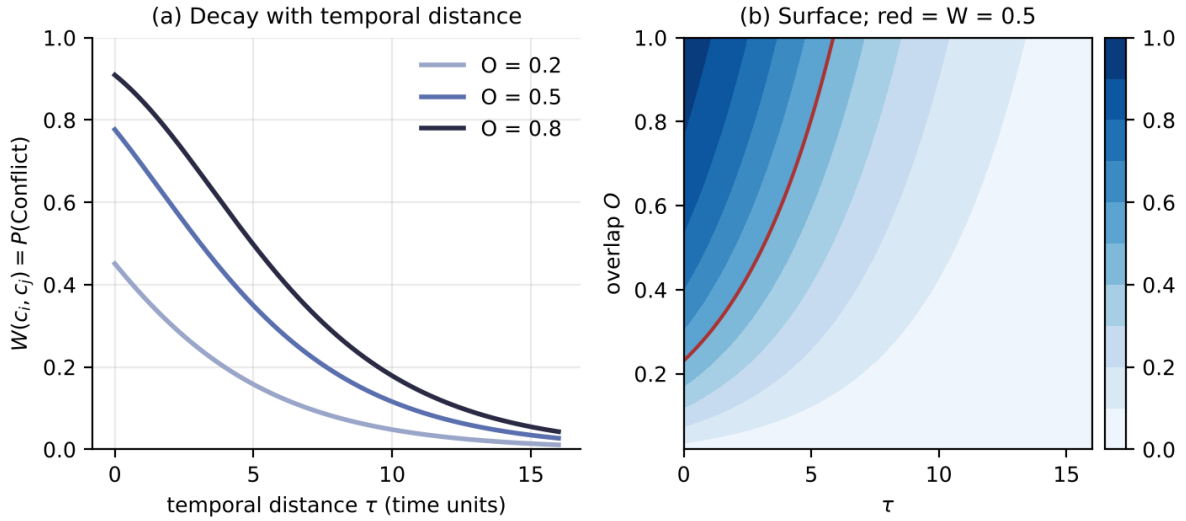


Figure 3. (a) The conflict window of Equation 1a as a function of temporal distance for three overlap levels; (b) its surface over (τ, O), with the W = 0.5 contour in red. Parameters $\kappa = 3$, $\theta = 4$ are illustrative. This is a plot of a formula, not a measurement.

Proposition 2 (monotonicity). Under Equation 1a, W is strictly decreasing in τ for O > 0, and strictly increasing in O for finite τ. *Proof.* Write $u = \kappa \cdot O \cdot e^{-\tau/\theta} > 0$. Since $W = 1 - e^{-u}$ is strictly increasing in u, and u is strictly decreasing in τ and strictly increasing in O, the claims follow. [QED]

3.2 Post-Merge Verification Operator (Definition 2)

Changesets bypass pre-merge queues and are appended directly to the trunk log. Verification is computed asynchronously and recursively:

$$V(c_i) = \oplus_V(\delta_i, \{ V(p) \mid p \in P(c_i) \}, CI(c_i)) \tag{2}$$

where $CI(c_i)$ is the set of build results, test results, and semantic assertion checks executed against the state at c_i. The result $V(c_i)$ takes one of three values: **verified**, **failed**, or **tainted**. The three-valued range is the key design decision. A two-valued operator that ANDs parent results would mark every descendant of a failed change as failed, making a single defect appear to be hundreds, and would send remediation agents after changesets that are not at fault.

An admissible instantiation. Let A(c) be the set of failing checks in CI(c). Define V(c) = *failed* if A(c) is non-empty and at least one failing check is attributable to δ (it fails on c but passes on every parent state); V(c) = *tainted* if A(c) is non-empty, no failure is attributable to δ, and some parent is failed or tainted; and V(c) = *verified* otherwise. Attribution is performed by re-running failing checks against the parent state, which is a bounded, parallelizable operation. Figure 4 shows the resulting propagation on a six-changeset

DAG, in which c3 — independent of the failing c2 — remains verified.

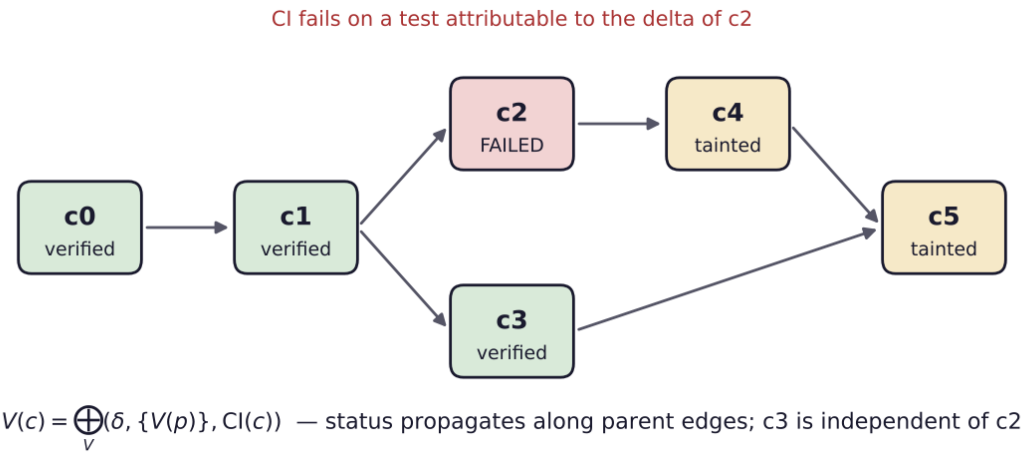


Figure 4. Recursive verification over a changeset DAG. c2 fails on a check attributable to its own delta; its descendants c4 and c5 are tainted, not failed; c3 is unaffected. Remediation targets c2 only. Illustrative diagram.

3.3 Auto-Remediation Bound (Definition 3)

When $V(c_i)$ = failed, a remediation agent is dispatched. Model the remediation subsystem as K tiers of independent attempts (for example: automatic revert, targeted patch, patch with broader context, semantic rewrite), each able to absorb the failure with some probability, with failures arriving at rate λ_{fail} and agents serving at aggregate capacity $\mu_{remediate}$. Then:

$$P(\text{Remediation}) \geq 1 - (\lambda_{fail} / \mu_{remediate})^{K} \cdot e^{-\gamma \cdot T_{react}} \quad (3)$$

Here $\rho = \lambda_{fail}/\mu_{remediate}$ is the remediation load ratio, K is the capacity tier depth, T_{react} is the *remediation time budget* — the interval before dependent work builds on the failed state and makes remediation costlier — and γ is a decay coefficient on residual risk per unit of that budget. (An interpretive note: the source draft glossed T_{react} as a "reaction time". Because Equation 3 improves as T_{react} grows, a time *budget* is the reading under which the formula behaves sensibly, and it is the reading used throughout.) As with the cache bound in AetherGit, Equation 3 is a structural property of the modeled system under steady-state assumptions, not a validated empirical law.

Proposition 3 (monotonicity and limit; minimum tier depth). If $\rho < 1$, the right-hand side of Equation 3 is non-decreasing in K and in T_{react}, and tends to 1 as K grows without bound. For a target residual risk $\varepsilon \in (0, 1)$, the bound is at least $1 - \varepsilon$ whenever

$$K \geq (\ln(1/\varepsilon) - \gamma \cdot T_{react}) / \ln(1/\rho) \quad (4)$$

Proof. The subtracted term $\rho^{K} e^{-\gamma T}$ is positive, decreasing in K when $0 < \rho < 1$, decreasing in T, and tends to 0 as K grows. Setting $\rho^{K} e^{-\gamma T} \leq \varepsilon$, taking logarithms, and dividing by $\ln(1/\rho) > 0$ yields Equation 4. [QED]

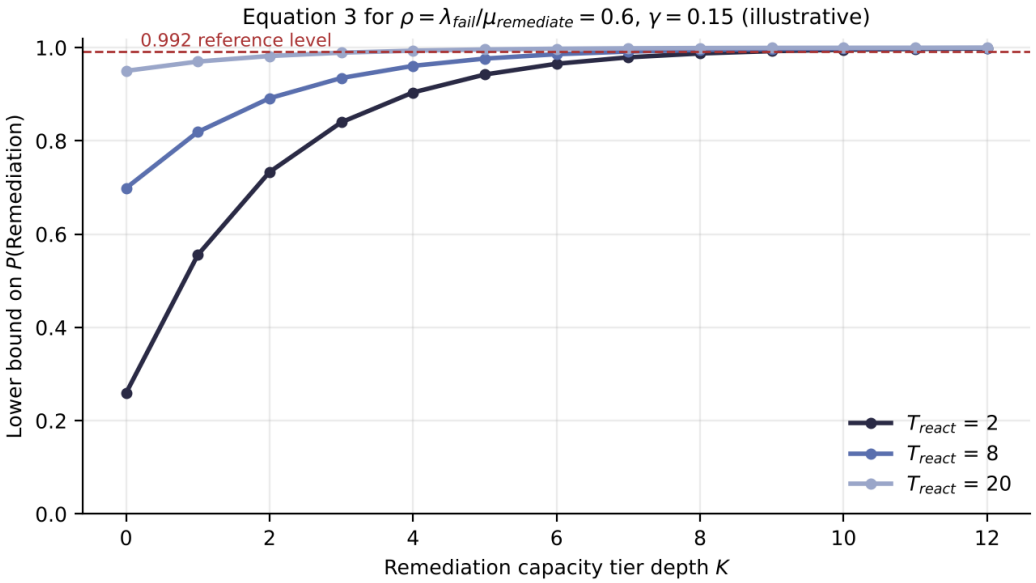


Figure 5. The Equation 3 lower bound versus tier depth K for three remediation time budgets, with ρ = 0.6 and γ = 0.15 (illustrative parameters). The dashed line marks the 0.992 level referenced in the source draft; this is a formula plot, not a measured success rate.

3.4 Why a Serial Merge Queue Saturates (Proposition 1)

Model the merge queue as a single server with Poisson arrivals at rate λ and exponential service time of mean s (the CI duration), so utilization is $\rho_q = \lambda s$. For $\rho_q < 1$ the M/M/1 mean time in system is

$$T_{queue} = s / (1 - \rho_q) \tag{5}$$

which diverges as ρ_q approaches 1 (Kleinrock, 1975), and for $\rho_q \geq 1$ the queue is unstable. In contrast, **lock-free entry** consists of one atomic append with cost a that does not depend on s or on the number of pending verifications.

Proposition 1 (admission independence). Under lock-free entry, the time from submission to admission to the trunk log is a, independent of the CI duration s and of the verification backlog. *Proof.* Admission consists only of the atomic append; verification is dispatched asynchronously after the append returns and no step of admission waits on it. [QED] *Caveat:* this moves the cost rather than eliminating it — verification still takes time s, and a failure is discovered after admission, not before. The architecture's claim is that this reordering is beneficial under the controls of Section 4, not that the work disappears.

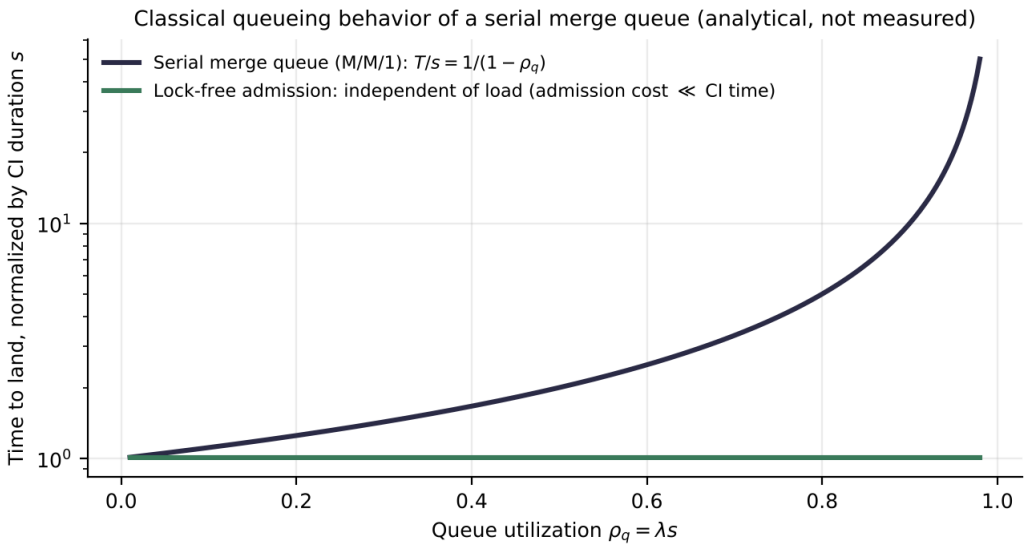


Figure 6. Analytical comparison of Equation 5 with a load-independent admission cost. The vertical axis is normalized by the CI duration s; the flat line assumes an admission cost of 1% of s, a hypothetical value. This is classical queueing analysis, not a measurement of any system.

4. System Architecture

AetherFlow is organized around three pillars and one consumption discipline. Figure 7 shows the lifecycle of a changeset through them.

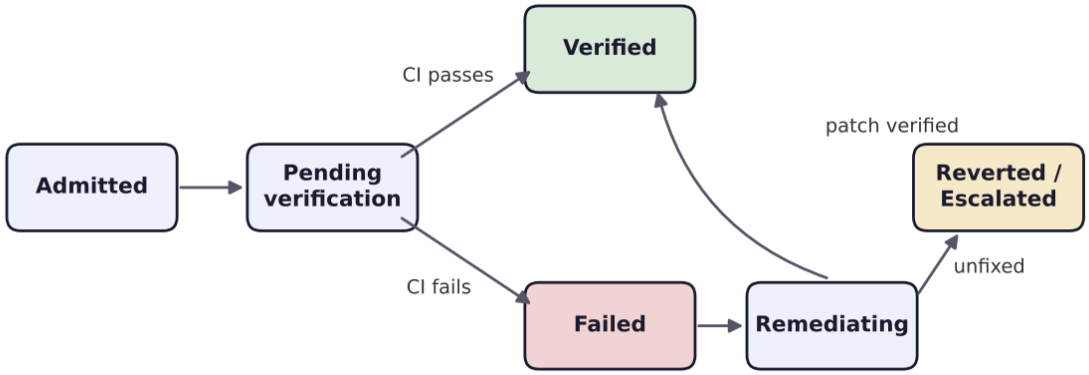


Figure 7. Changeset lifecycle. Admission precedes verification; failures enter a bounded remediation loop that terminates in verified state, revert, or human escalation. Illustrative state diagram.

4.1 Pillar 1 — Lock-Free Merge Entry

Every incoming agentic changeset is appended to the trunk's reference log by an atomic compare-and-swap on the log head, retried on contention. No queue, branch-protection gate, or global lock stands between submission and admission. Because changesets are content-addressed and carry their own parent pointers (Baladi, 2026b), concurrent appends do not need to rebase one another: the log orders them, and the DAG records what each author had seen. The dynamic conflict window (Definition 1) is evaluated at admission as a *hint* — it prioritizes verification of high-W pairs — and never as a gate.

4.2 Pillar 2 — Post-Merge Verification Layer

Immediately after admission, verification workers execute builds, tests, and semantic dependency scans against the new state. Work is scheduled in priority order: changesets with high conflict-window scores against recent neighbours, changesets touching widely depended-upon code (high out-degree in G), and changesets whose parents are not yet verified. Failing checks trigger attribution (Section 3.2), producing the failed/tainted distinction that the remediation layer consumes. Verification is decentralized: workers are stateless with respect to the log and can be added or removed without coordination.

4.3 The Verified Reference: Consuming a Lock-Free Trunk Safely

The principal risk of lock-free entry is that the trunk head may be temporarily unverified. AetherFlow confines that risk by distinguishing two references: the **admission head**, which advances on every append, and the **verified head**, defined as the most recent changeset all of whose ancestors are verified. Producers branch from, and agents read context from, the admission head so that they see the latest work. Anything that requires a known-good state — releases, deployments, downstream consumers, human review views — reads the verified head. The invariant the design maintains is therefore not “the trunk is always green” but “the verified head is always green”; the first invariant is what pre-merge gating buys at the price of serialization, and the second is what AetherFlow buys at the price of a bounded window of admitted-but-unverified state.

4.4 Pillar 3 — Proactive Remediation Agents

When a changeset is marked failed, a remediation agent is dispatched with the failing changeset, the failing checks, and the last verified state. The policy is graded, with each tier corresponding to one of the K tiers of Equation 3:

Tier	Action	Typical trigger	Compensation type
1	Automatic revert of the failing changeset	Failure is clearly attributable, no dependents yet	Full compensation (saga undo)
2	Targeted micro-patch	Small, local failure with an identifiable cause	Forward correction
3	Patch with broader context	Failure involves interaction with adjacent changes	Forward correction
4	Semantic conflict resolution / rewrite	Two intents that cannot both be preserved textually	Forward correction, higher risk
Final	Escalation to a human owner	Tiers exhausted or guardrail triggered	Human decision

Guardrails. Because the remediating party is an agent, remediation must be constrained so that it cannot satisfy the verifier by weakening it. A corrective patch is rejected if it deletes or disables the failing test, modifies test infrastructure, widens an assertion's tolerance, or changes files outside a declared blast radius derived from the failing changeset and its dependency neighbourhood — unless a human approves. A remediation attempt is itself a changeset subject to the same verification, and a bounded retry budget prevents unbounded remediation loops.

4.5 Integration with the AetherGit Storage Layer

AetherFlow's operators are defined over the AetherGit changeset log. The verification operator $\oplus_V$ has the same recursive structure as AetherGit's derived-data operator (Baladi, 2026b, Equation 3): both compute a property of a changeset from its own delta and the property values of its parents. A verification status can therefore be stored and served as one more derived-data category, cached under the same tiered topology, and invalidated by the same mechanism when a remediation changeset is appended. This is a convenient implementation path rather than a requirement; any store exposing an append-only DAG would suffice.

5. Illustrative Worked Example (Case Study)

> ***Status:*** *This section evaluates the formulas of Section 3 at hypothetical parameters chosen for exposition. It is not a benchmark, a simulation, or a measurement of a deployed system, and the parameters are not claimed to match any real workload.*

5.1 Scenario

Consider a hypothetical organization in which a fleet of autonomous agents submits 30,000 changesets per day to one monorepo trunk, an arrival rate of λ = 30,000 / 86,400 ≈ 0.347 changesets per second. Suppose the full CI pipeline for a changeset takes s = 20 minutes (1,200 s).

5.2 The Serial Queue Under This Load

For a single serial queue, utilization is $\rho_q = \lambda s = 0.347 \times 1{,}200 \approx 417$. Because ρ_q is far above 1, Equation 5 does not apply: the queue is unstable and its backlog grows without bound. To be stable, a serial design would need roughly 418 or more parallel verification lanes, plus a way to reconcile them — which is exactly what speculative merge systems attempt (Ananthanarayanan et al., 2019). The calculation illustrates the scaling pressure, not the performance of any particular product; real systems batch, parallelize, and trim CI scope in ways this arithmetic ignores.

5.3 Conflict Window for Three Pairs of Concurrent Changesets

Using Equation 1a with κ = 3 and θ = 4 time units:

Pair	Overlap O	Temporal distance τ	W = P(Conflict)	Reading
A	0.5	1	0.689	Nearby code, injected almost together: likely mutually unaware
B	0.5	10	0.116	Same overlap, injected much later: likely informed by earlier change
C	0.8	0.5	0.880	Heavy overlap, near-simultaneous: highest verification priority

Pairs A and C would be placed at the front of the verification schedule (Section 4.2); pair B would not. Nothing is blocked in any case.

DOI: https://doi.org/10.5281/zenodo.23979129

5.4 Remediation Depth for a Target Residual Risk

Take $\rho = \lambda_{fail}/\mu_{remediate} = 0.6$, $\gamma = 0.15$, $T_{react} = 8$, and a target residual risk $\varepsilon = 0.008$ (that is, a bound of 99.2%). Equation 4 gives $K \geq (\ln(1/0.008) - 0.15\times8) / \ln(1/0.6) \approx 7.10$, so $K = 8$ tiers suffice. At $K = 4$ the bound is 0.961; at $K = 8$ it is 0.995. If the time budget were negligible ($T_{react} = 0$), the requirement rises to $K \geq 9.45$, i.e. $K = 10$. The time budget therefore substitutes for roughly 2 tiers of capacity in this example — a structural statement about the formula, and the origin of the practical advice that remediation should begin immediately.

5.5 Walk-Through: One Failing Changeset

1. Agent X submits c2 modifying a function signature; agent Y concurrently submits c3 in an unrelated module. Both are admitted at once (Proposition 1).
2. Verification of c2 fails on a unit test that passes at c1 — attribution marks c2 **failed**. A later changeset c4 built on c2 also fails the same test; no failure is attributable to its own delta, so c4 is **tainted**. c3 verifies normally (Figure 4).
3. The verified head stays at c1 (or c3's branch, if its ancestors are all verified); consumers reading the verified head never see c2.
4. Tier 1 dispatch: revert of c2 is not applicable because c4 depends on it. Tier 2: a targeted patch updating the call site passes the full check set, and the patch is itself verified before the verified head advances.
5. Had no tier succeeded, c2 would have been reverted together with dependents held for resubmission, and a human owner notified.

5.6 Reading the Example

The example shows three things and no more: that a serial queue is structurally unable to absorb agent-scale arrival rates without large parallelism; that the conflict window concentrates verification effort where conflicts are probable; and that the remediation bound gives a closed-form capacity requirement. Whether any real workload exhibits parameters of these magnitudes is an empirical question deferred to Section 7.

6. Comparative Positioning

Table 1 positions AetherFlow's specified properties against three integration models. Entries for existing systems follow the cited public documentation; entries for AetherFlow reflect this paper's specification and not a measured implementation.

Property	Git + merge queue	Speculative gating	Pre-submit + post-submit CI	AetherFlow (specified)
Verification placement	Before merge	Before merge, speculative	Both (checks before, TAP after)	After admission
Admission blocks on CI	Yes	Yes (parallelized)	Pre-submit checks only	No (Prop. 1)
Trunk invariant	Always green	Always green	Green at pre-submit; breakage handled post-submit	Verified head green; admission head may be unverified
Failure attribution	Per queued change	Per speculative build	Culprit finding	Recursive, three-valued (Def. 2)
Descendants of a failure	Not applicable (never merged)	Not applicable	Handled by triage	Marked tainted, not failed

DOI: https://doi.org/10.5281/zenodo.23979129

Property	Git + merge queue	Speculative gating	Pre-submit + post-submit CI	AetherFlow (specified)
Repair actor	Author / human	Author / human	Human / tooling	Remediation agents with guardrails
Formal bound on repair	None	None	None	Eq. 3–4 (structural, unvalidated)

The rightmost column is deliberately phrased as specification. In particular, the comparison does not assert that AetherFlow outperforms the alternatives: it states where the designs differ structurally. The trade AetherFlow makes — a window of admitted-but-unverified state in exchange for admission independence — is a genuine cost, and for some organizations (for example those with strict regulatory release controls on every trunk commit) it may be unacceptable.

7. Proposed Evaluation Protocol

The source draft stated expected improvements as a comparison table. Those values — an integration latency of 30–120 minutes for merge queues versus 0.5–2.0 seconds for AetherFlow (≈100×), zero queue starvation, and a remediation success rate of at least 99.2% — are retained here only as **design targets**, not as results. Table 2 distinguishes what is structural from what is a target to be tested.

Metric	Merge-queue baseline	AetherFlow	Status
Submission-to-admission latency	Grows with queue depth (Eq. 5)	Atomic append cost a; independent of s	Structural (Prop. 1); magnitude untested
Submission-to-verified latency	Queue wait + CI	CI duration s, plus remediation if failed	Untested; may be comparable
Queue starvation	Possible near $\rho_q = 1$	No queue	Structural
Hypothesized speed-up (source draft: ~100×)	30–120 min	0.5–2.0 s	Design target; unvalidated
Remediation success (source draft: ≥ 99.2%)	Human review dependent	Eq. 3 bound	Design target; K = 8 suffices in the stated example

Note on the two latency measures. Admission latency and verified latency are different quantities. The headline benefit of lock-free entry concerns the first; an evaluation that reports only admission latency would overstate the benefit if verified latency is not also reported.

7.1 Workloads, Baselines, and Metrics

- **Workloads:** (i) replay of a real repository's commit history at accelerated arrival rates; (ii) synthetic agent-generated changesets with controlled footprint overlap; (iii) injected-fault workloads with known culprits for attribution accuracy.
- **Baselines:** a Git-based merge queue; a speculative-gating configuration; a pre-submit-plus-post-submit configuration.
- **Primary metrics:** admission latency, verified latency, fraction of time the admission head is unverified, attribution precision/recall, remediation success rate per tier, count of guardrail violations attempted by remediation agents.
- **Ablations:** two-valued versus three-valued verification; with and without conflict-window prioritization; $K = 1 \ldots K_{max}$ tiers; guardrails off (to measure how often agents weaken tests).

7.2 Validating Equations 1a and 3

Equation 1a's parameters κ, θ, L can be fit by logistic-style regression on observed conflicts versus (τ, O); the quality of fit is then a direct test of the functional form. Equation 3 is tested by estimating λ_{fail}, $\mu_{remediate}$, and the per-tier success probabilities from logs and comparing the predicted bound to the observed remediation success rate across load levels; a bound that is violated falsifies the independence assumption behind it.

8. Discussion and Threats

8.1 What Formalization Adds

Post-submit testing, optimistic concurrency, and compensation are individually established. What the formalization adds is a single vocabulary in which three otherwise informal claims can be checked: that admission is independent of verification (Proposition 1), that verification status propagates without blame inflation (Definition 2), and that remediation capacity can be sized from a target residual risk (Proposition 3).

8.2 The Red-Trunk Window

Lock-free entry makes the admission head transiently unverified. The verified-head discipline (Section 4.3) confines the effect to producers, but two costs remain. First, agents building on an admitted-but-failed changeset create tainted work that must later be re-verified; the remediation time budget T_{react} of Equation 3 is the formal expression of this cost. Second, human collaborators browsing the admission head may see broken code; interfaces should default to the verified head.

8.3 Flaky Tests and Attribution Errors

Flaky tests (Luo et al., 2014) can produce spurious failures and mislead attribution: a flaky failure could be attributed to an innocent changeset, triggering needless remediation, or could mask a real failure. Mitigations include re-running failing checks against the parent state a bounded number of times, quarantining tests with a history of nondeterminism, and treating attribution confidence as an input to the remediation policy. None of these is evaluated here.

8.4 Semantic Conflicts Invisible to Tests

A conflict that no test detects — a semantic incompatibility between two changes that individually and jointly pass the suite — is outside the reach of any CI-based verifier, pre-merge or post-merge (Brun et al., 2011). AetherFlow neither worsens nor solves this; its contribution is to make conflict-window scores available as a signal for targeted deeper analysis where overlap is high.

8.5 Remediation Agents as a Security and Integrity Surface

An agent permitted to edit the trunk in response to failure is a privileged actor. Beyond the guardrails of Section 4.4, deployments should authenticate remediation changesets, record their provenance in the log, rate-limit them, and make every remediation reversible. A compromised or misaligned remediation agent is a risk that pre-merge human review mitigates and post-merge automation can amplify.

DOI: https://doi.org/10.5281/zenodo.23979129

8.6 When Not to Use This Architecture

The design is not appropriate where an unverified trunk commit has unacceptable consequences, where test suites are too slow or too unreliable for attribution to be trusted, or where arrival rates are low enough that a conventional merge queue has no meaningful backlog. In those regimes, the complexity of AetherFlow is not justified.

9. Limitations

1 AetherFlow has not been implemented. No claim in this paper should be read as reporting measured performance of a running system.

2 Section 5 uses hypothetical parameters chosen for exposition; they are not derived from, and should not be read as approximating, any real system's arrival rate, CI duration, λ_{fail}, $\mu_{remediate}$, K, or T_{react}.

3 The function f of Definition 1 is left abstract; Equation 1a is one admissible instantiation, chosen for monotonicity and tractability, and has not been fit to data.

4 Equation 3's steady-state and independence assumptions are standard simplifications that have not been validated against any agentic workload trace; correlated failures (for example a shared infrastructure outage) would violate them.

5 The operator $\oplus_V$ is specified structurally and instantiated only through the three-valued rule of Section 3.2; attribution by re-execution is assumed cheap and reliable.

6 The 100× latency and 99.2% remediation figures from the source draft are design targets only (Section 7); no analysis here supports them as results.

7 Comparative positioning relies on public documentation and was not verified against any system's source code.

8 The claim of formal rigor extends only to internal consistency among Definitions 1–3 and Propositions 1–3 as stated.

10. Conclusion and Future Work

AetherFlow argues that scaling integration for autonomous agents calls for separating admission from verification: append immediately, verify asynchronously, attribute failures recursively, repair with graded and guarded automation, and let consumers read a verified head. The formal contributions — a conflict window, a three-valued recursive verification operator, and a remediation bound with a closed-form minimum tier depth — make these design choices checkable. Within the stated assumptions, admission latency is independent of CI duration, and remediation capacity can be sized from a residual-risk target.

Future work, in order of priority: (1) implement the admission and verification layers over an AetherGit-style log and execute the protocol of Section 7; (2) fit and test Equations 1a and 3 on real commit and CI histories; (3) study attribution accuracy under flaky tests; (4) evaluate guardrail effectiveness against test-weakening behaviour by remediation agents; and (5) explore adaptive scheduling of verification using the conflict window.

DOI: https://doi.org/10.5281/zenodo.23979129

Appendix A — Notation

Symbol	Meaning
$c_i = (\delta_i, P(c_i), \tau_i, \sigma_i)$	Changeset: delta, parents, timestamp, content-addressed signature (Baladi, 2026b)
$\tau(c_i, c_j)$	Temporal distance between injections of two changesets
G	Dependency graph of the codebase
$O \in [0, 1]$	Overlap score from file-footprint overlap and dependency-graph proximity
κ, θ	Sensitivity and decay horizon of Equation 1a
$W(c_i, c_j)$	Dynamic conflict window: probability of conflict
$V(c), \oplus_V$	Verification status (verified / failed / tainted) and its recursive operator
CI(c)	Build, test, and semantic-check results executed at c
λ_{fail}	Rate of post-merge integration failures
$\mu_{remediate}$	Aggregate processing capacity of remediation agents
$\rho = \lambda_{fail}/\mu_{remediate}$	Remediation load ratio
K	Remediation capacity tier depth
T_{react}, γ	Remediation time budget and residual-risk decay coefficient
ε	Target residual risk
λ, s, ρ_q	Arrival rate, CI duration, and utilization of a serial merge queue
a	Cost of the atomic append performed at admission

Appendix B — Derivations

B.1 Minimum tier depth (Equation 4)

From $\rho^K e^{-\gamma T} \leq \varepsilon$: $K \ln \rho - \gamma T \leq \ln \varepsilon$. Since $\ln \rho < 0$, divide by $\ln \rho$ and reverse the inequality: $K \geq (\ln \varepsilon + \gamma T)/\ln \rho = (\ln(1/\varepsilon) - \gamma T)/\ln(1/\rho)$.

B.2 M/M/1 time in system (Equation 5)

For arrival rate λ and service rate $\mu = 1/s$ with $\lambda < \mu$, the mean time in system is $1/(\mu - \lambda) = s/(1 - \lambda s)$ (Kleinrock, 1975). By Little's law (Little, 1961), the mean number in system is λ times this quantity, which likewise diverges as λs approaches 1.

B.3 Numerical values in Section 5

With $\kappa = 3$, $\theta = 4$: $W(1, 0.5) = 1 - \exp(-3 \cdot 0.5 \cdot e^{-0.25}) = 0.689$; $W(10, 0.5) = 0.116$; $W(0.5, 0.8) = 0.880$. With $\rho = 0.6$, $\gamma = 0.15$, $T = 8$: $e^{-1.2} = 0.301$; the bound at $K = 8$ is $1 - 0.6^8 \cdot 0.301 = 0.995$.

References

1. Ananthanarayanan, S., Ardekani, M. S., Haenikel, D., et al. (2019). **Keeping Master Green at Scale.** Proceedings of the Fourteenth EuroSys Conference. https://doi.org/10.1145/3302424.3303970
2. Anthropic. (2026). **Redesigning Source Control at Anthropic.** Anthropic Research. https://www.anthropic.com/research/redesigning-source-control
3. Baladi, S. (2026a). **Rethinking Source Control for Agentic Software Engineering: Architectural Transformations, Data Models, and Scalability Constraints in AI-Driven Development.** Zenodo. https://doi.org/10.5281/zenodo.23934465
4. Baladi, S. (2026b). **AetherGit: Scalable Source Control for Autonomous AI Agents.** Zenodo. https://doi.org/10.5281/zenodo.23966896
5. Brun, Y., Holmes, R., Ernst, M. D., & Notkin, D. (2011). **Proactive Detection of Collaboration Conflicts.** Proceedings of ESEC/FSE 2011. https://doi.org/10.1145/2025113.2025139
6. Chacon, S., & Straub, B. (2014). **Pro Git** (2nd ed.). Apress. https://git-scm.com/book
7. Fowler, M. (2006). **Continuous Integration.** https://martinfowler.com/articles/continuousIntegration.html
8. Garcia-Molina, H., & Salem, K. (1987). **Sagas.** Proceedings of the 1987 ACM SIGMOD Conference. https://doi.org/10.1145/38713.38742
9. Herlihy, M. (1991). **Wait-Free Synchronization.** ACM Transactions on Programming Languages and Systems, 13(1), 124–149. https://doi.org/10.1145/114005.102808
10. Jimenez, C. E., Yang, J., Wettig, A., et al. (2024). **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?** ICLR 2024. arXiv:2310.06770.
11. Kleinrock, L. (1975). **Queueing Systems, Volume 1: Theory.** Wiley-Interscience.
12. Kung, H. T., & Robinson, J. T. (1981). **On Optimistic Methods for Concurrency Control.** ACM Transactions on Database Systems, 6(2), 213–226. https://doi.org/10.1145/319566.319567
13. Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). **GenProg: A Generic Method for Automatic Software Repair.** IEEE Transactions on Software Engineering, 38(1), 54–72. https://doi.org/10.1109/TSE.2011.104
14. Little, J. D. C. (1961). **A Proof for the Queuing Formula: L = λW.** Operations Research, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383
15. Luo, Q., Hariri, F., Eloussi, L., & Marinov, D. (2014). **An Empirical Analysis of Flaky Tests.** Proceedings of the 22nd ACM SIGSOFT FSE. https://doi.org/10.1145/2635868.2635920
16. Potvin, R., & Levenberg, J. (2016). **Why Google Stores Billions of Lines of Code in a Single Repository.** Communications of the ACM, 59(7), 78–87. https://doi.org/10.1145/2854146
17. Sapling SCM. (n.d.). **Sapling: A Scalable, User-Friendly Source Control System.** Meta Platforms. https://sapling-scm.com/

Reference entries were assembled from the author's knowledge of the cited works and should be checked against the publishers' records before formal submission; references 2, 3, 4, and 17 are the sources for system-specific claims about Scion, the author's prior work, and Meta's tooling.

Research Status Statement

Component	Current status
Architecture specification (three pillars + verified-head discipline)	Specified (Sections 3–4)
Formal definitions (Definitions 1–3)	Specified (Section 3)
Propositions 1–3 and proofs	Stated with proofs (Section 3, Appendix B)
Illustrative worked example	Presented; hypothetical parameters, not measured (Section 5)
Comparative positioning	Presented from public documentation (Section 6)
Implementation of AetherFlow	Not built
Benchmark / simulation / empirical measurement	Not performed
Speed-up and success-rate figures from source draft	Retained only as design targets (Section 7)
Evaluation protocol	Proposed (Section 7); not executed
Relation to prior work	Companion to Baladi (2026b); sequel line from Baladi (2026a)

Record ID: AETHERFLOW-01 · Version 1.0.0 · DOI: https://doi.org/10.5281/zenodo.23979129
Scope: formal architecture and specification paper with illustrative analysis. No implementation or benchmark reported.
Next research stage: implement the admission and verification layers; execute the evaluation protocol of Section 7.

Samir Baladi · Independent Interdisciplinary Researcher · Ronin Institute / Rite of Renaissance · ORCID: 0009-0003-8903-0029
gitdeeper@gmail.com
© 2026 Samir Baladi — MIT License